

JSON en C# y ASP.NET Core – Internet & HTTP

Luis Antonio Castillo Javier

Tiempo estimado: 3 hrs



Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Escuela de Ingeniería en Ciencias y Sistemas

Segundo Semestre 2026

[IPC-2 - Introducción a la Programación y Computación 2 - Sección D]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en esta lectura?
Responsabilidad	Al consumir APIs externas con HttpClient, el desarrollador es responsable de manejar correctamente los errores y códigos HTTP, garantizando que la aplicación no falle silenciosamente.
Integridad	Usar HTTPS y TLS asegura que los datos transmitidos entre cliente y servidor no sean alterados ni interceptados, reflejando integridad en la comunicación.
Innovación	La integración de Razor Pages con ASP.NET Core Web API mediante JSON representa una arquitectura moderna y eficiente para el desarrollo de aplicaciones web.

1.2 Competencia(s)

Tipo de Competencia	Descripción
Competencia General	Desarrollar aplicaciones web usando JSON en el contexto de C# y ASP.NET Core.
Competencia Específica 1	Analiza respuestas de peticiones web en C# mediante HttpClient y las librerías System.Text.Json / Newtonsoft.Json para obtener datos seguros desde APIs externas.
Competencia Específica 2	Implementa comunicación cliente-servidor con Razor Pages como frontend y ASP.NET Core Web API como backend, usando HTTP/HTTPS para APIs seguras y eficientes.

2. Palabras Clave

JSON (JavaScript Object Notation): Formato de texto ligero para el intercambio de datos basado en pares clave-valor, ampliamente utilizado en APIs REST.

Serialización: Proceso de convertir un objeto C# en una cadena de texto JSON para su transmisión o almacenamiento.

Deserialización: Proceso inverso: convertir una cadena JSON de regreso a un objeto C# tipado.

HttpClient: Clase de .NET para enviar peticiones HTTP y recibir respuestas desde servicios web o APIs REST.

Códigos de respuesta HTTP: Números de tres dígitos que el servidor devuelve para indicar el resultado de una petición.

ASP.NET Core Web API: Framework de Microsoft para construir APIs REST de alto rendimiento sobre .NET.

Razor Pages: Modelo de programación de ASP.NET Core para construir páginas web dinámicas en C# con archivos .cshtml.

HTTPS/TLS: Protocolo de comunicación segura que encripta los datos entre cliente y servidor mediante certificados digitales.

3. Introducción

En el desarrollo moderno de aplicaciones web, la capacidad de intercambiar datos entre sistemas de forma eficiente y segura es fundamental. JSON se ha consolidado como el estándar de facto para este intercambio, siendo la columna vertebral de las APIs REST que impulsan la web actual. En el ecosistema de .NET, tanto System.Text.Json como Newtonsoft.Json ofrecen herramientas poderosas para serializar y deserializar objetos C# hacia y desde JSON.

Esta lectura cubre dos grandes bloques temáticos de la Semana 11 del curso IPC-2: primero, los fundamentos de JSON en C# y su comparación con XML; y segundo, la comunicación cliente-servidor mediante HTTP/HTTPS usando HttpClient, códigos de respuesta y la integración entre Razor Pages y ASP.NET Core Web API. Dominar estos conceptos permite construir arquitecturas web robustas, seguras y mantenibles.

4. Contenidos

Presentación de temas y subtemas desarrollados en la lectura.

Tema 1: ¿Qué es JSON y por qué es importante en .NET?

JSON es un formato de texto ligero para el acceso, almacenamiento e intercambio de datos. Es la alternativa más popular a XML y el estándar de facto para APIs REST. En el ecosistema .NET existen dos librerías principales para trabajar con JSON:

- **System.Text.Json**: Librería nativa de .NET 6/7/8. Alta performance, ideal para ASP.NET Core Web API.
- **Newtonsoft.Json**: Paquete NuGet muy popular. Muy flexible y compatible con proyectos legacy.

JSON resulta fundamental para ASP.NET Core Web API por cuatro razones principales: es ligero y rápido, permite serialización automática de objetos C# sin configuración extra, es universal porque cualquier cliente HTTP puede consumirlo y soporta estructuras complejas como listas y objetos anidados.

Subtema 1.1: Sintaxis JSON

La sintaxis JSON sigue reglas sencillas y estrictas: los datos se organizan en pares clave-valor, los elementos se separan por comas, las llaves agrupan objetos y los corchetes agrupan arreglos. Las claves deben ir entre comillas dobles y los valores pueden ser string, number, object, array, boolean o null.

Subtema 1.2: JSON vs XML en .NET

Característica	JSON (System.Text.Json)	XML (System.Xml)
Origen	Creado en 2001 por Douglas Crockford	Publicado en 1998 por W3C
Estructura	Pares Clave-Valor	Árbol jerárquico con tags
Sintaxis	Compacta y fácil de leer	Detallada con etiquetas de apertura/cierre
Parseo en .NET	JsonSerializer.Deserialize<T>() >()	XmlDocument o XmlSerializer
Tamaño	Archivos más pequeños → menor latencia	Archivos más voluminosos

Uso recomendado

REST APIs y Razor Pages

Documentos complejos y
sistemas legacy**Tabla 1 - Comparativa JSON vs XML en .NET****Tema 2: Tipos de Datos y Serialización/Deserialización**

El mapeo entre tipos JSON y tipos C# es directo y consistente. Comprender esta equivalencia es esencial para trabajar con APIs REST de forma efectiva.

Tipo JSON	Tipo C#	Ejemplo JSON	Anotación
string	string	"Laptop Pro"	[JsonPropertyName]
number	int / decimal	1299.99	[JsonPropertyName]
boolean	bool	true / false	[JsonPropertyName]
null	null / nullable	null	[JsonPropertyName]
object	class / record	{ "id": 1 }	[JsonPropertyName]
array	List<T> / T[]	["a","b","c"]	[JsonPropertyName]

Tabla 2 - Mapeo de tipos JSON a tipos C#**Subtema 2.1: Serialización con System.Text.Json vs Newtonsoft.Json**

Ambas librerías permiten convertir objetos C# a JSON y JSON a objetos C#. La diferencia principal radica en el rendimiento y la flexibilidad.

- System.Text.Json: JsonSerializer.Serialize(objeto) y JsonSerializer.Deserialize<Tipo>(json). Es la opción recomendada para nuevos proyectos .NET 6+.
- Newtonsoft.Json: JsonConvert.SerializeObject(objeto) y JsonConvert.DeserializeObject<Tipo>(json). Ofrece más opciones de formato como Formatting.Indented para JSON legible.

Tema 3: Internet, HTTP y HttpClient en C#

HTTP sigue el modelo cliente-servidor sin estado. Es el protocolo base de la World Wide Web y define cómo se comunican los clientes con los servidores. En .NET, la clase HttpClient es la herramienta principal para enviar peticiones HTTP y consumir APIs REST desde aplicaciones como Razor Pages.

Subtema 3.1: Verbos HTTP en C#

Verbo HTTP	Método HttpClient	Uso principal
GET	GetAsync(url)	Obtener recursos del servidor
POST	PostAsJsonAsync(url, obj)	Crear un nuevo recurso
PUT	PutAsJsonAsync(url, obj)	Actualizar un recurso existente completo
DELETE	DeleteAsync(url)	Eliminar un recurso
PATCH	PatchAsJsonAsync(url, obj)	Actualizar parcialmente un recurso

Tabla 3 - Verbos HTTP y métodos HttpClient en .NET

Subtema 3.2: Códigos de Respuesta HTTP

Los códigos HTTP son números de tres dígitos que el servidor devuelve para indicar el resultado de cada petición. En ASP.NET Core, los controladores retornan estos códigos usando métodos de IActionResult.

Familia	Significado	Ejemplos en ASP.NET Core
1XX	Informativos – petición recibida	Continue, SwitchingProtocols
2XX	Éxito – petición procesada correctamente	Ok(), Created(), NoContent()
3XX	Redirección – recurso movido	RedirectToAction(), Redirect()
4XX	Error del cliente – petición incorrecta	BadRequest(), NotFound(), Unauthorized()

5XX	Error del servidor – fallo interno	StatusCode(500), Problem()
-----	------------------------------------	----------------------------

Tabla 4 - Familias de códigos de respuesta HTTP

Tema 4: HTTPS y Seguridad en ASP.NET Core

HTTPS es la versión segura de HTTP, encriptada con TLS/SSL. En ASP.NET Core se configura automáticamente con el servidor Kestrel. Utiliza criptografía asimétrica: la clave pública cifra la sesión inicial y la clave privada, que reside en el servidor, descifra los datos.

Los elementos clave de seguridad HTTPS en ASP.NET Core son:

- HSTS (HTTP Strict Transport Security): Se activa con `app.UseHsts()` para forzar que todos los clientes usen HTTPS.
- Redirección automática: `app.UseHttpsRedirection()` redirige todo el tráfico HTTP al puerto HTTPS.
- Autenticación y autorización: Se configura con `app.UseAuthentication()` y `app.UseAuthorization()` respectivamente.

Caso de estudio

Considera una tienda en línea donde el frontend Razor Pages necesita mostrar un catálogo de productos obtenido desde un backend ASP.NET Core Web API. Cuando el usuario visita la página de productos, Razor Pages llama al método `OnGetAsync()`, usa `HttpClient` para hacer un GET al endpoint del API y este responde con un 200 OK y un arreglo JSON de productos.

Figura 1 - Ejemplo integrador: Razor Pages consumiendo un Web API con JSON

Backend: ASP.NET Core Web API

```
[ApiController]
[Route("api/[controller]")]
public class ProductosController :
    ControllerBase
{
    [HttpGet]
    public IActionResult GetAll()
    {
        var lista = _service.GetAll();
        return Ok(lista);
    }
}
```

Frontend: Razor Pages

```
public class ProductosModel : PageModel
{
    public List<Producto> Items { get; set; }

    public async Task OnGetAsync()
    {
        Items = await
            _http.GetFromJsonAsync<List<Producto>>(
                "https://api/productos");
    }
}
```




En este caso, `System.Text.Json` deserializa automáticamente la respuesta a una `List<Producto>` de C#, que luego se renderiza en el HTML con un bucle `@foreach`. Esto muestra cómo JSON funciona como puente entre el backend y el frontend dentro de una arquitectura web moderna.

5. Conclusiones

JSON es el formato estándar para el intercambio de datos en aplicaciones web modernas. En el ecosistema .NET, System.Text.Json ofrece alto rendimiento para nuevos proyectos, mientras que Newtonsoft.Json sigue siendo valioso por su flexibilidad en proyectos legacy. La elección entre JSON y XML debe basarse en el contexto de uso.

La comunicación cliente-servidor con HttpClient, junto al correcto uso de los códigos de respuesta HTTP y la implementación de HTTPS desde el inicio, forma la base de cualquier aplicación web robusta, segura y mantenible. La arquitectura Razor Pages + ASP.NET Core Web API, unida por JSON, representa una solución moderna y escalable para el desarrollo web con .NET.

6. Referencias

1. Microsoft. (2024). System.Text.Json overview. Microsoft Learn.
<https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview>
2. Microsoft. (2024). ASP.NET Core Web API. Microsoft Learn.
<https://learn.microsoft.com/en-us/aspnet/core/web-api>
3. Microsoft. (2024). Razor Pages in ASP.NET Core. Microsoft Learn.
<https://learn.microsoft.com/en-us/aspnet/core/razor-pages>
4. MDN Web Docs. (2025). Códigos de estado HTTP.
<https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Status>
5. MDN Web Docs. (2025). HTTP. <https://developer.mozilla.org/es/docs/Web/HTTP>
6. Newtonsoft. (2024). Json.NET Documentation.
<https://www.newtonsoft.com/json/help/html/Introduction.htm>
7. GeeksforGeeks. (2025). HTTP Full Form. <https://www.geeksforgeeks.org/http-full-form/>